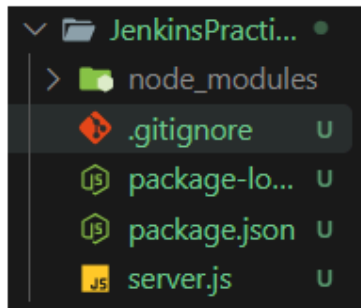


Aim : To automate the deployment of the project using the Jenkins

Here for the demonstration of the tool we are creating the project in node.js and express

Procedure:

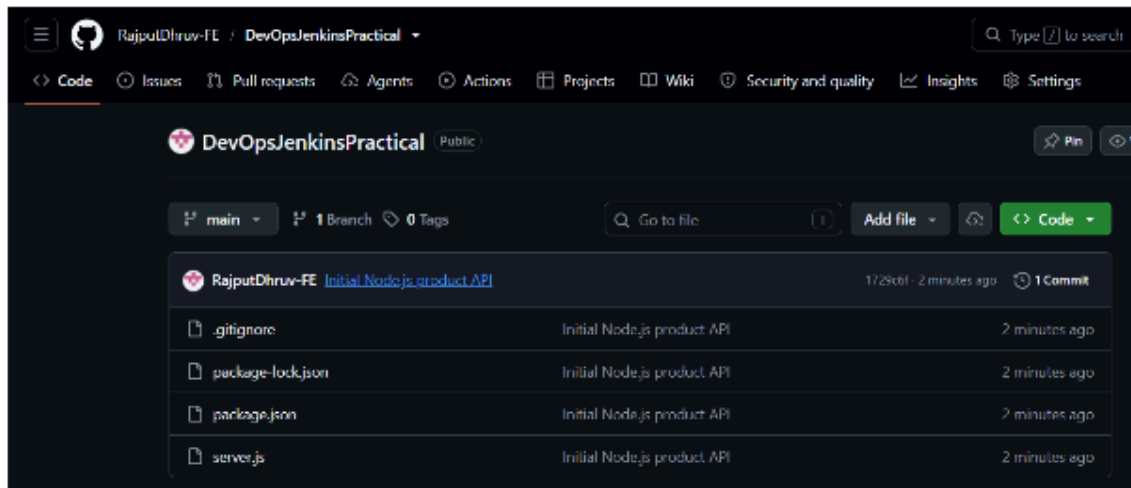
Step 1: Create the project as given below



Step 2: Initialize the project with git and commit the initial setup

- git init
- git add .
- git commit -m "Initial Node.js product API"

Step 3 : Create GitHub repo and push the commits



Step 4: Dockerize the Application

- Create the "Dockerfile"

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]

- Verify the application by creating the image and running the container
 - o `docker build -t node-products-api .`
 - o `docker run -d -p 3000:3000 --name products-api node-products-api`
- Push the Docker configuration files to the Github

Step 5: Setup Jenkins

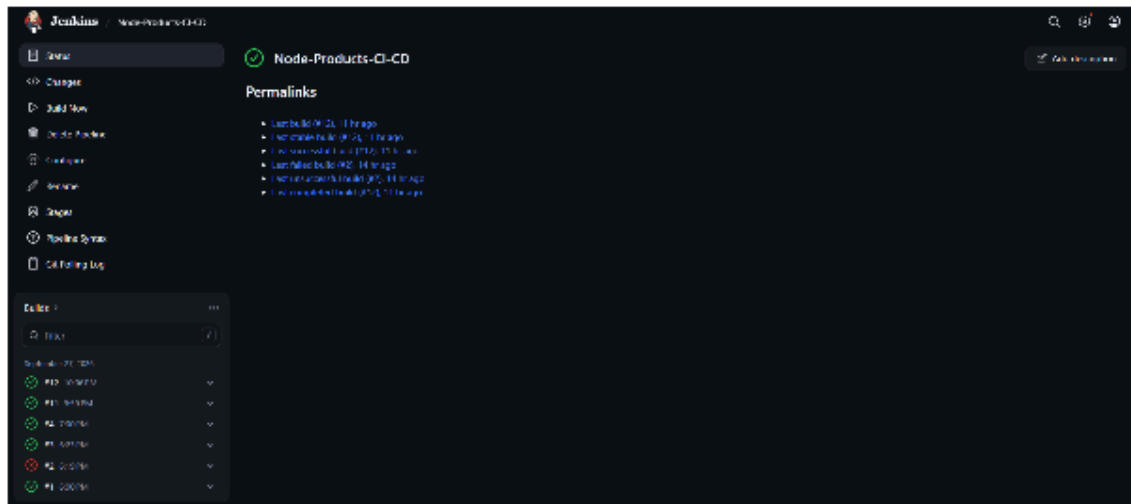
Jenkins is used to automate the CI/CD process of the Node.js Product API. In this practical, Jenkins is connected with the GitHub repository and Docker Hub. Poll SCM is used to automatically detect new commits pushed to GitHub.

5.1 Start Jenkins

Start Jenkins on the local system and open the Jenkins dashboard in a web browser.

`http://localhost:8080`

Log in to the Jenkins dashboard using the configured administrator account.

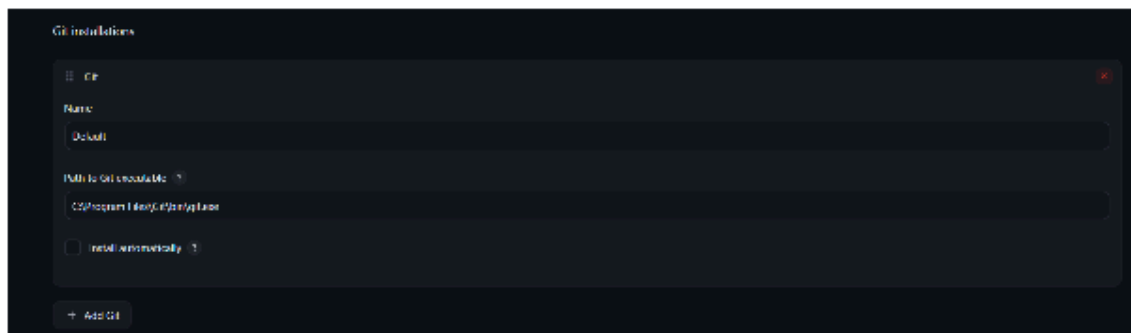


5.2 Configure Git in Jenkins

Open Jenkins and go to Manage Jenkins → Tools. Configure Git so that Jenkins can execute Git commands.

The Git executable used in this practical is:

`C:\Program Files\Git\bin\git.exe`

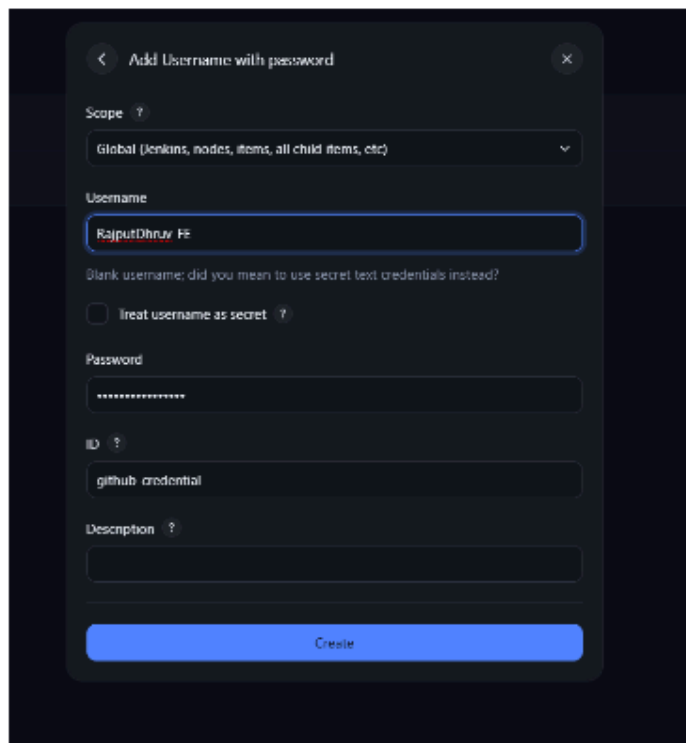


5.3 Add GitHub Credentials

Go to Manage Jenkins → Credentials and add the GitHub credential required to access the repository.

Credential ID:

`github-credentials`



The image shows a dark-themed dialog box titled "Add Username with password". It contains the following fields and options:

- Scope**: A dropdown menu with the selected value "Global (Jenkins, nodes, items, all child items, etc)".
- Username**: A text input field containing "RajputDhruv FF".
- A warning message: "Blank username; did you mean to use secret text credentials instead?"
- ☐ **Treat username as secret**
- Password**: A text input field with masked characters (dots).
- ID**: A text input field containing "github-credential".
- Description**: An empty text input field.
- A blue **Create** button at the bottom.

5.4 Add Docker Hub Credentials

Add a Docker Hub credential so that Jenkins can log in to Docker Hub and push the Docker image.

Credential ID:

dockerhub-credentials

Docker Hub username:

dhruvddu

The password/token field contains the Docker Hub Personal Access Token.

< Add Username with password X

Scope ?
Global (Jenkins, nodes, items, all child items, etc) v

Username
dhruvddu

☐ Treat username as secret ?

Password

ID ?
dockerhub-credential

Description ?

Create

5.5 Create a Jenkins Pipeline Job

From the Jenkins dashboard, select New Item.

- Enter the item name: Node-Products-CI-CD
- Select Pipeline.
- Click OK.

Jenkins

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue v

No builds in the queue.

Build Executor Status ^

(0 of 2 executors busy)

New Item

Enter an item name
Node-Products-CI-CD

Select item type

Pipeline
Build, test and deploying pipelines. Supports reuse, parallel work, and running on multiple agents.

Freestyle project
Classic, general purpose job type that divides out from up to one SCM repository build step usually followed by post-build steps for archiving artifacts and sending email notifications.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
Creates a container for folders called for Jenkins. Useful for grouping things together. Folders, which is just a folder in Jenkins, creates a container namespace for you with however many things of the same name as long as they are in different folders.

Multi-branch pipeline
Creates a set of Pipelines, projects working in parallel based on one SCM repository.

Organization folder
Creates a set of multi-branch pipelines with one file workspace for repositories.

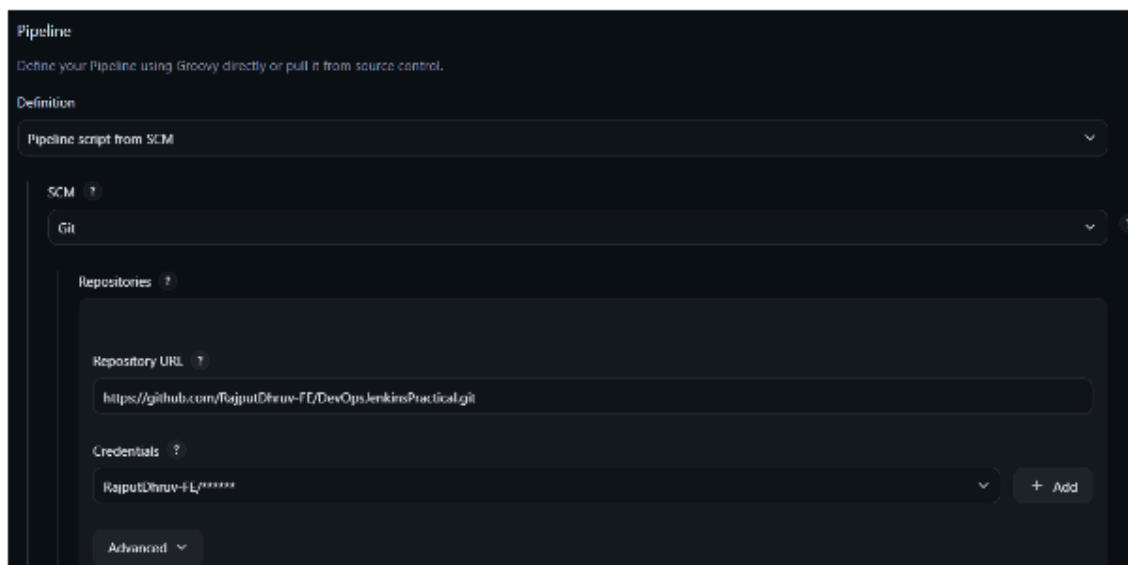
OK

5.6 Configure Pipeline from SCM

In the job configuration, select Pipeline script from SCM so Jenkins obtains the

Jenkinsfile directly from the GitHub repository.

- Definition: Pipeline script from SCM
- SCM: Git
- Repository URL: <https://github.com/RajputDhruv-FE/DevOpsJenkinsPractical.git>
- Credentials: github-credentials
- Branch Specifier: */main
- Script Path: Jenkinsfile



Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

<https://github.com/RajputDhruv-FE/DevOpsJenkinsPractical.git>

Credentials ?

RajputDhruv-FE/*****

+ Add

Advanced

5.7 Jenkinsfile

Create a file named Jenkinsfile in the project root. The Jenkinsfile defines the CI/CD stages.

- Install dependencies using npm install.
- Run tests using npm test.
- Build the Docker image.
- Log in to Docker Hub using Jenkins credentials.
- Push the Docker image to Docker Hub.
- Remove the old container and deploy the new container.
- Verify the Product API.

Commit and push the Jenkinsfile to the GitHub repository.

5.8 Configure Poll SCM

Poll SCM is used to periodically check the GitHub repository for changes. This is the automatic triggering method used in this practical because Jenkins is running locally.

Go to:

Jenkins → Node-Products-CI-CD → Configure → Build Triggers

Enable:

- Poll SCM

Enter the schedule:

* * * * *

This makes Jenkins check the Git repository every minute for new changes.

The GitHub webhook trigger is not used in this practical.

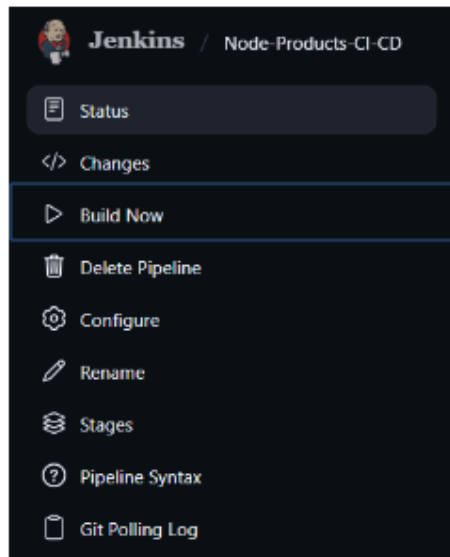


5.9 Run the Initial Build

Click Save and select Build Now for the initial execution of the pipeline.

Jenkins executes the following stages:

Checkout → npm install → npm test → Docker Build → Docker Hub Login
→ Docker Push → Deploy → Verify API



5.10 Verify the Pipeline

Open the generated build and view the pipeline stages. All stages should complete successfully.

The final status should be:

Finished: SUCCESS

```
[Pipeline] }  
[Pipeline] // afterEnv  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```

5.11 Test Automatic Build Using Poll SCM

Make a small change to server.js. For example, modify the message returned by the root endpoint.

message: "Product API is running successfully through Jenkins CI/CD"

Save and push the change to GitHub:

```
git add .  
git commit -m "Update API message"  
git push
```

Do not click Build Now. Jenkins detects the new commit during the next SCM polling cycle and automatically starts a new build.

5.12 Verify the Updated Deployment

After the automatic build completes successfully, verify that the Docker container is running.

```
docker ps
```

Verify the Product API:

```
curl http://localhost:3000/products
```

Verify the updated root response:

```
curl http://localhost:3000/
```

Final CI/CD Flow

```
Developer
  ↓ git push
GitHub Repository
  ↓ Poll SCM
Jenkins
  ↓
Checkout
  ↓
npm install
  ↓
npm test
  ↓
Docker Build
  ↓
Docker Hub Push
  ↓
Deploy Container
  ↓
Verify Product API
```

↓
SUCCESS